# Лабораторная работа 1

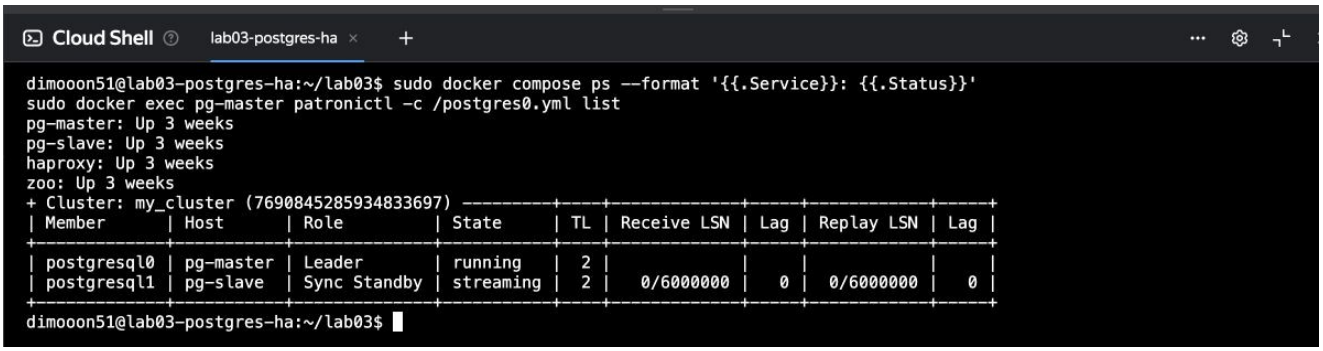
Высокодоступный кластер PostgreSQL

### Цель работы

Развернуть кластер PostgreSQL и проверить репликацию и автоматическую смену лидера.

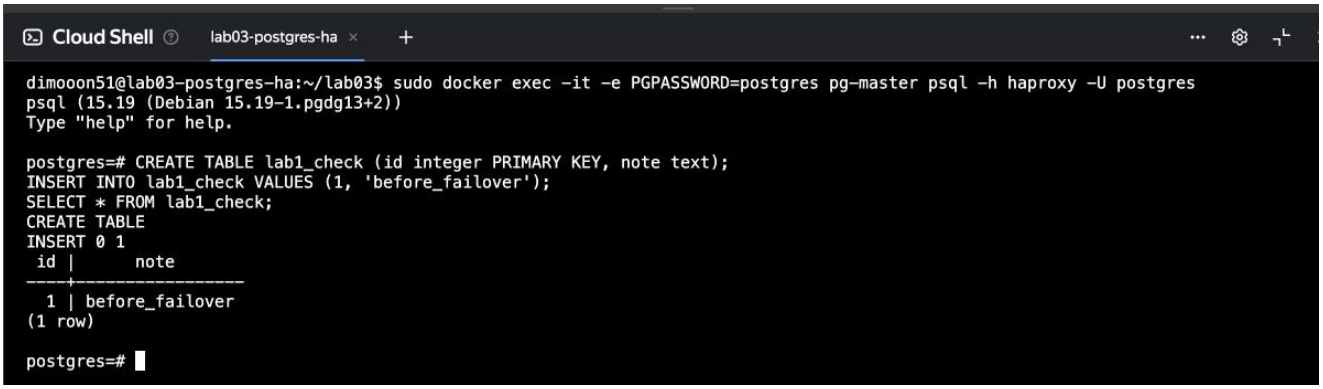
### 1 Запуск кластера

На BM Ubuntu 24.04 запущены два узла PostgreSQL 15 с Patroni, ZooKeeper и HAProxy. pg-master работает как Leader, pg-slave — как Sync Standby. Отставание реплики равно 0. Стенд проверяет отказ узла PostgreSQL; одна BM, один ZooKeeper и один HAProxy остаются единичными точками отказа.



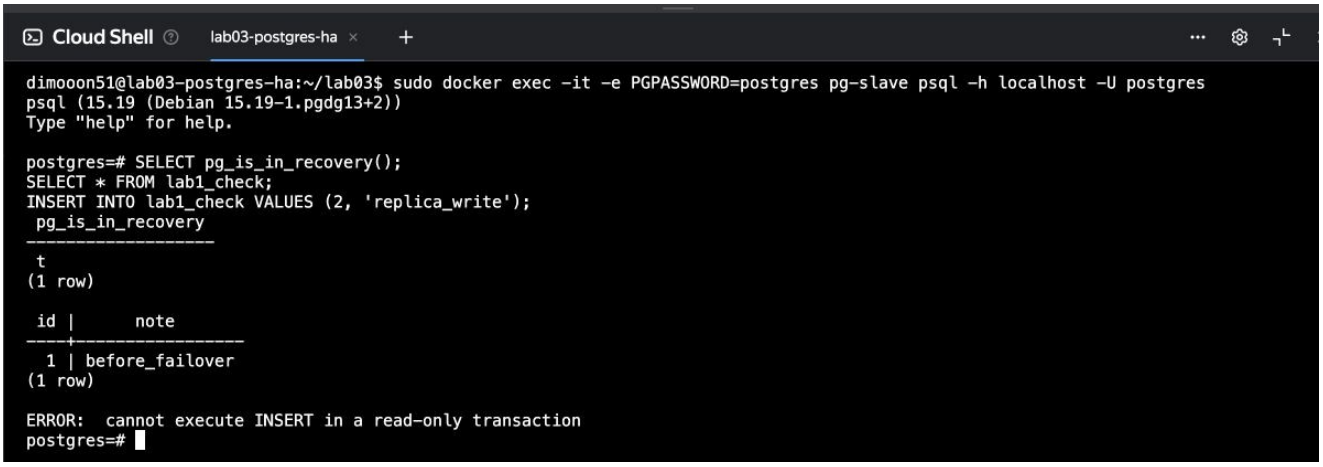

### 2 Создание таблицы

Через haproxy:5432 создали таблицу lab1_check и добавили строку before_failover.



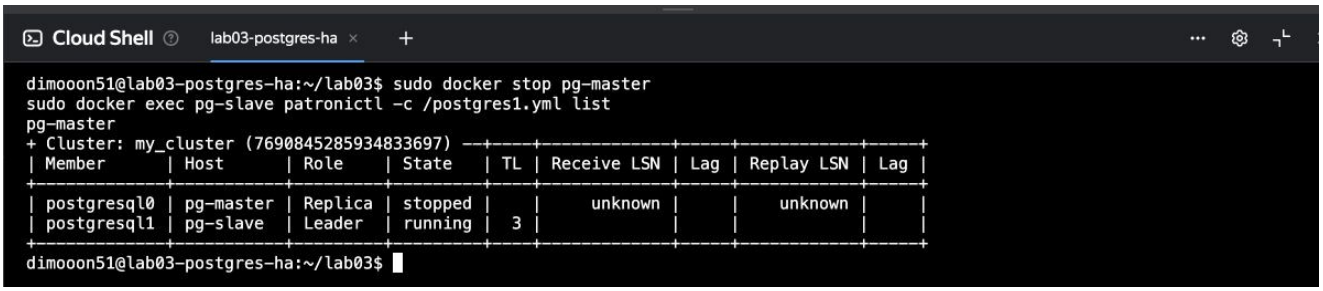

### 3 Проверка репликации

На pg-slave появилась та же строка. pg_is_in_recovery() вернул true. Запись на реплику запрещена: INSERT завершился ошибкой read-only transaction.

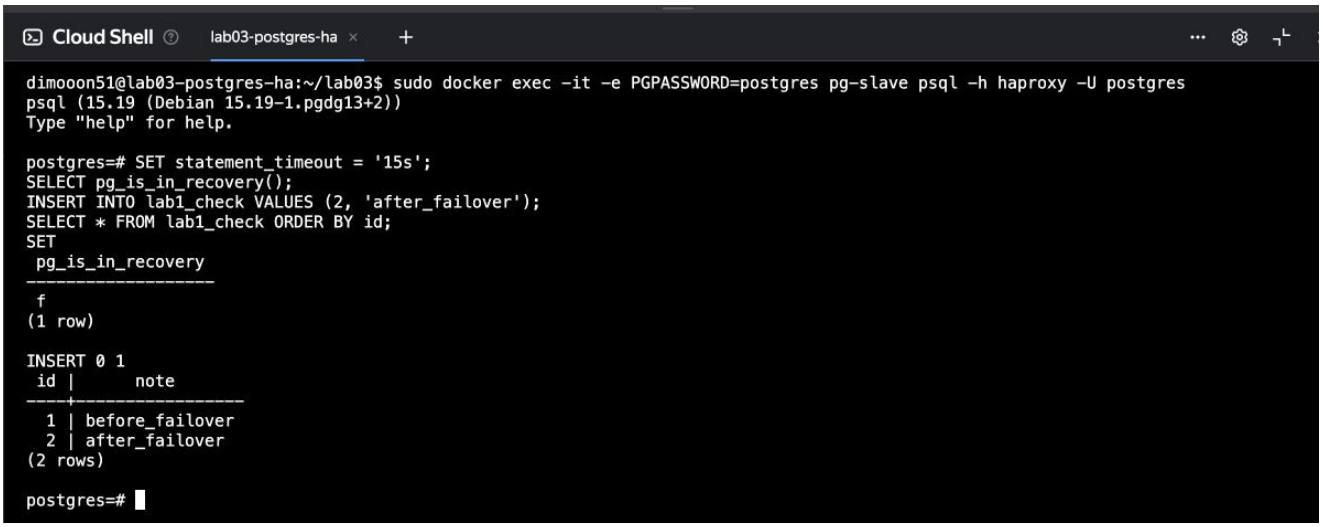


### 4 Остановка лидера

После docker stop pg-master узел pg-slave стал лидером. Timeline изменилась с 2 на 3.

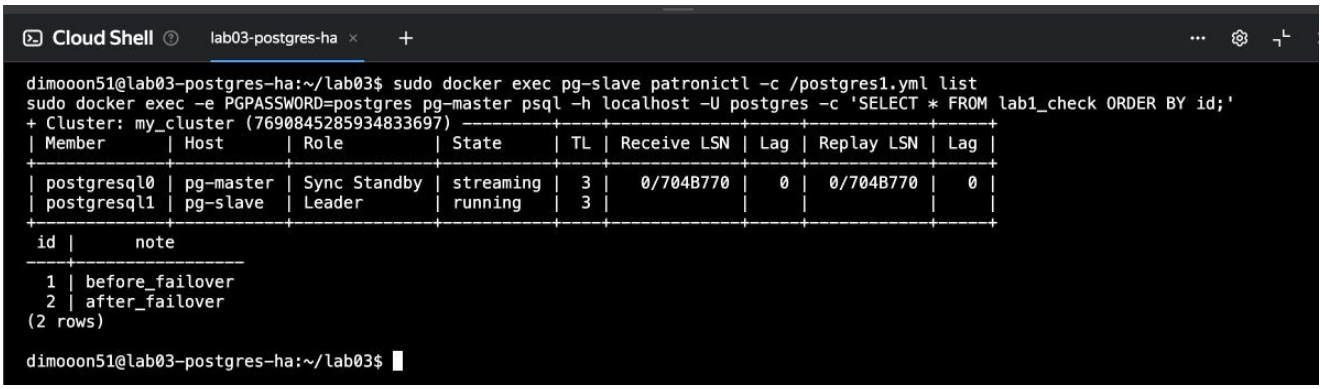



Пока pg-master выключен, через тот же адрес HAProxy добавили строку after_failover и прочитали обе строки.




### 5 Возврат узла

После docker start pg-master узел вернулся как Sync Standby с отставанием 0. Обе строки появились на нём автоматически.




### 6 Ответы на вопросы

**Чем отличаются expose и ports?**

expose не публикует порт на хосте; ports публикует его, например 5433:5432. В общей Docker-сети контейнеры доступны друг другу и без expose.

**Нужна ли пересборка после изменения Dockerfile или postgresX.yml?**

После изменения Dockerfile или скопированных postgresX.yml нужен docker compose up -d --build; restart образ не пересобирает. Параметры bootstrap.dcs задаются при создании кластера; позже их меняют через patronictl edit-config.

### Вывод

Репликация и автоматическая смена лидера работают. После переключения чтение и запись доступны через прежний адрес HAProxy. Вернувшийся узел получает пропущенные данные.